

Java Full Stack Development

3. Working with Copilot





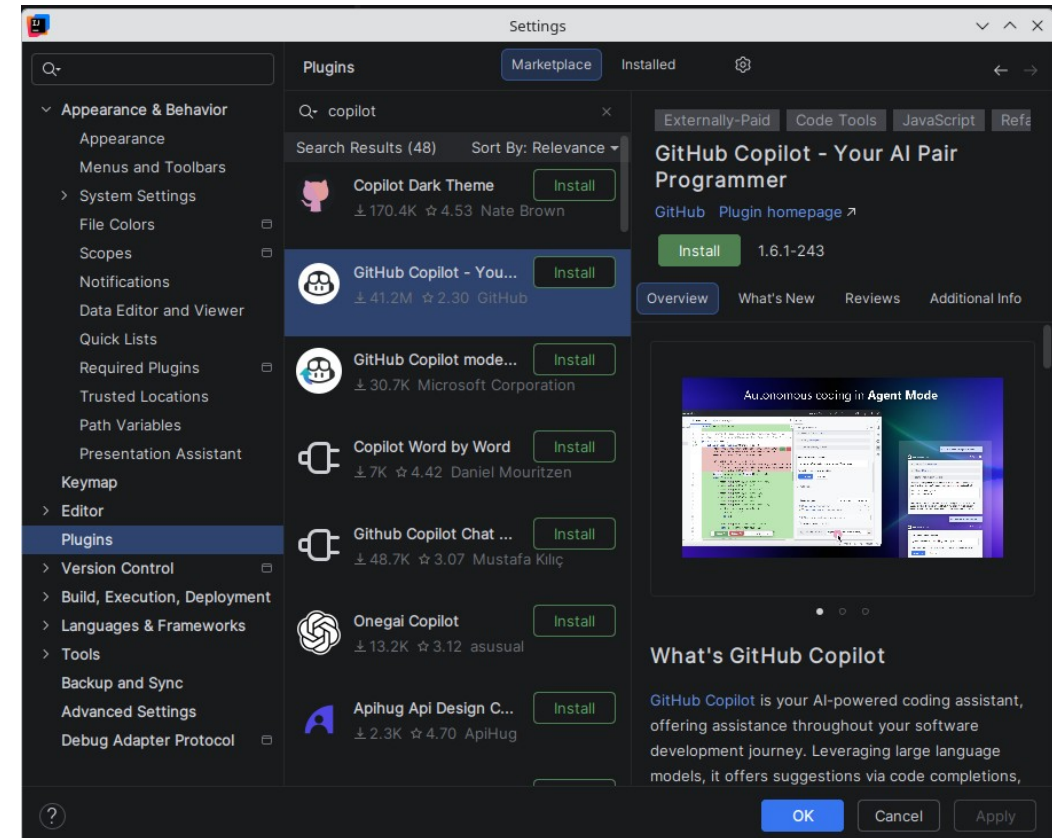
Module Topics

- Copilot IntelliJ Integration
- Understanding Copilot
- How Prompts Work
- Using Copilot to Write and Validate Code
- Prompt Engineering

Copilot IntelliJ Integration

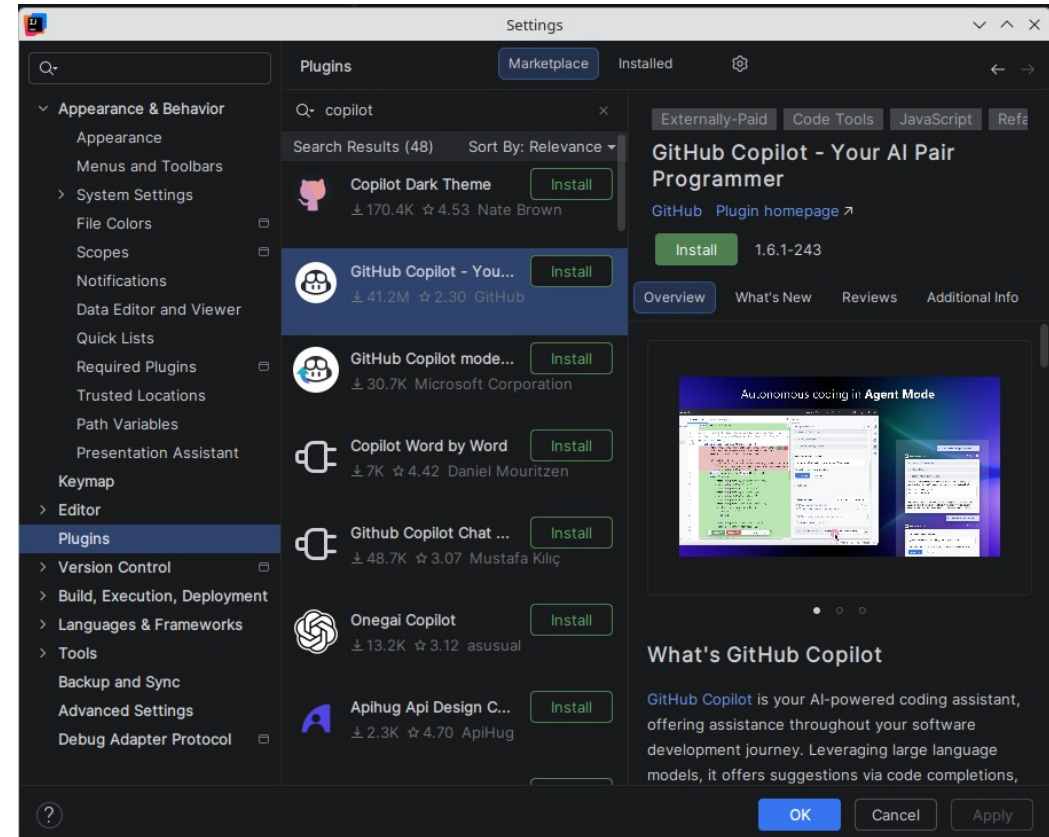
Copilot Integration

- GitHub Copilot integrates into IntelliJ
 - Via first-party JetBrains plugin
 - Developed and maintained by GitHub
- Available for IntelliJ IDEA
 - Both Community and Ultimate editions



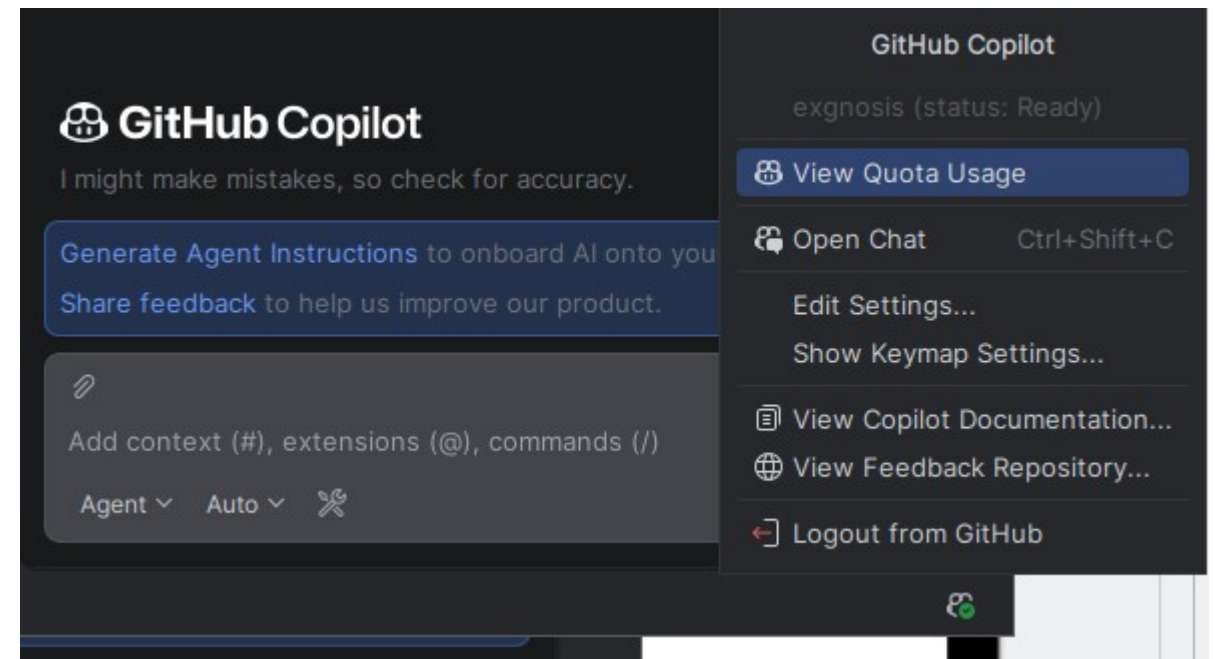
Copilot Integration

- Open up **Plugins** in settings
 - Search for GitHub Copilot
 - Install the plugin published by GitHub
 - Restart IntelliJ to activate
- Post-install
 - Sign in to GitHub
 - **Tools → GitHub Copilot → Login to GitHub**
 - A browser window opens for OAuth authentication
 - The IDE receives an access token on successful login



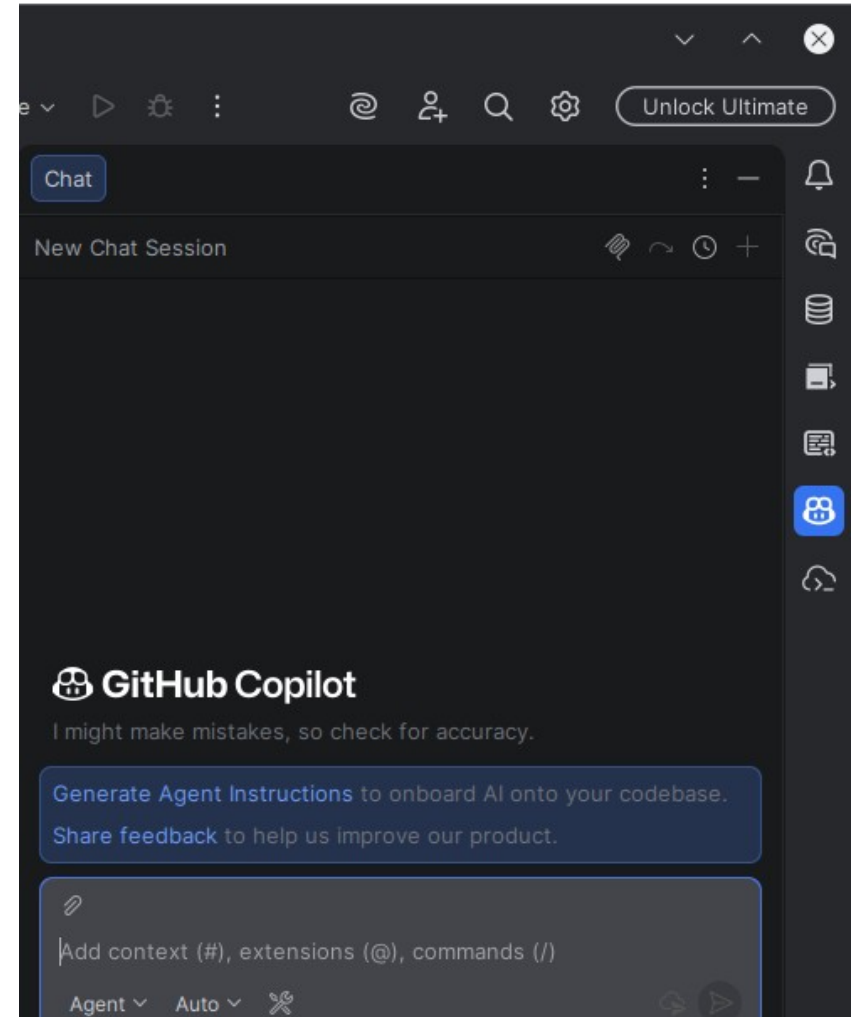
Copilot Integration

- Verify installation
 - Copilot icon appears in the status bar
 - Bottom right of the IDE
 - Status bar shows active/inactive state



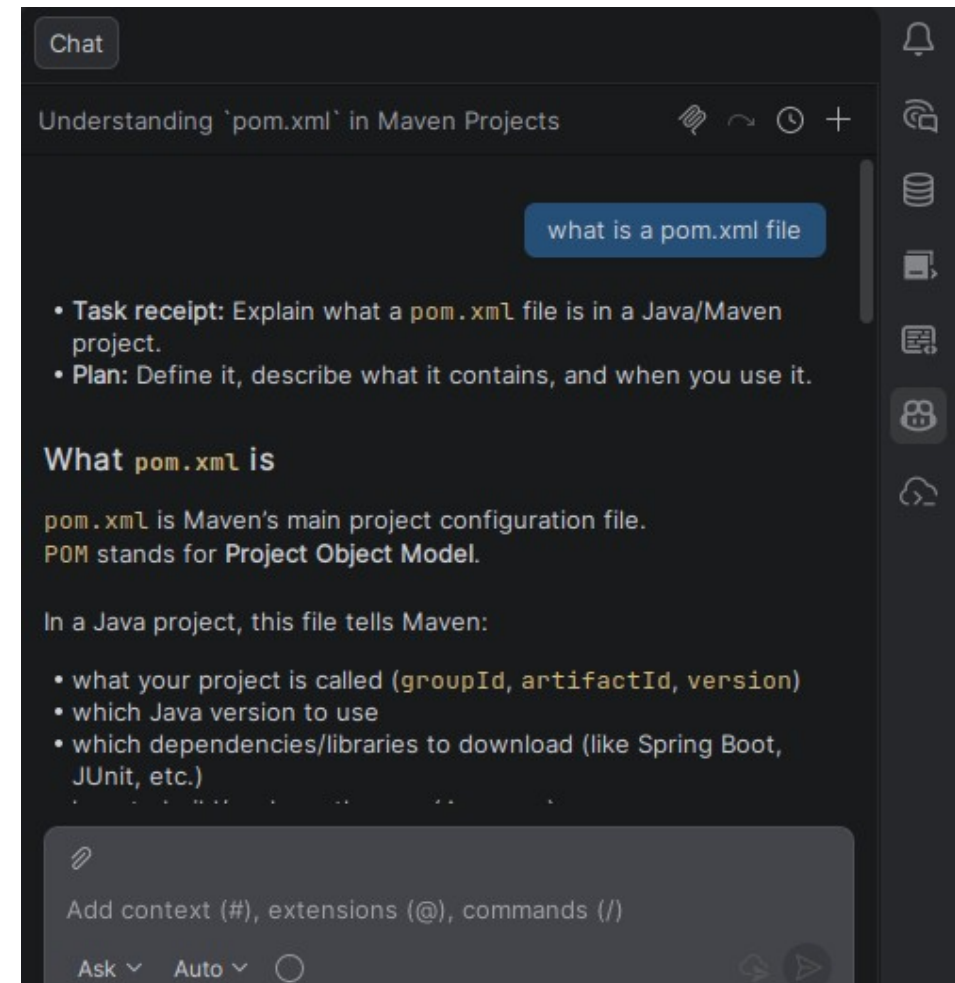
Copilot Integration

- Once installed, Copilot surfaces in two primary modes
 - Inline completions
 - Suggestions appear directly in the editor as you type
 - Copilot chat
 - A conversational interface docked inside the IDE
 - Both modes are available without leaving the editor
- Requires a GitHub Copilot license
 - Individual, Business, or Enterprise plan
 - Authentication is via your GitHub account



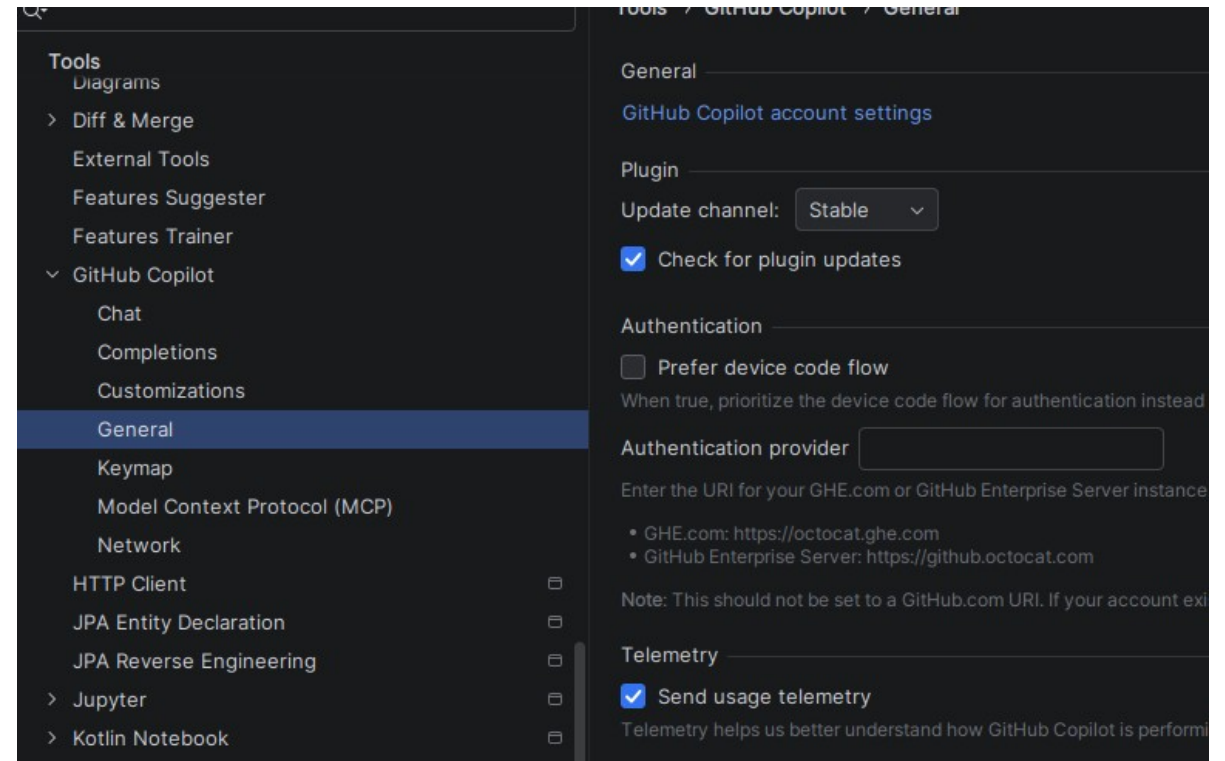
Copilot Integration

- Using chat mode is interactive
 - Shown in sidebar



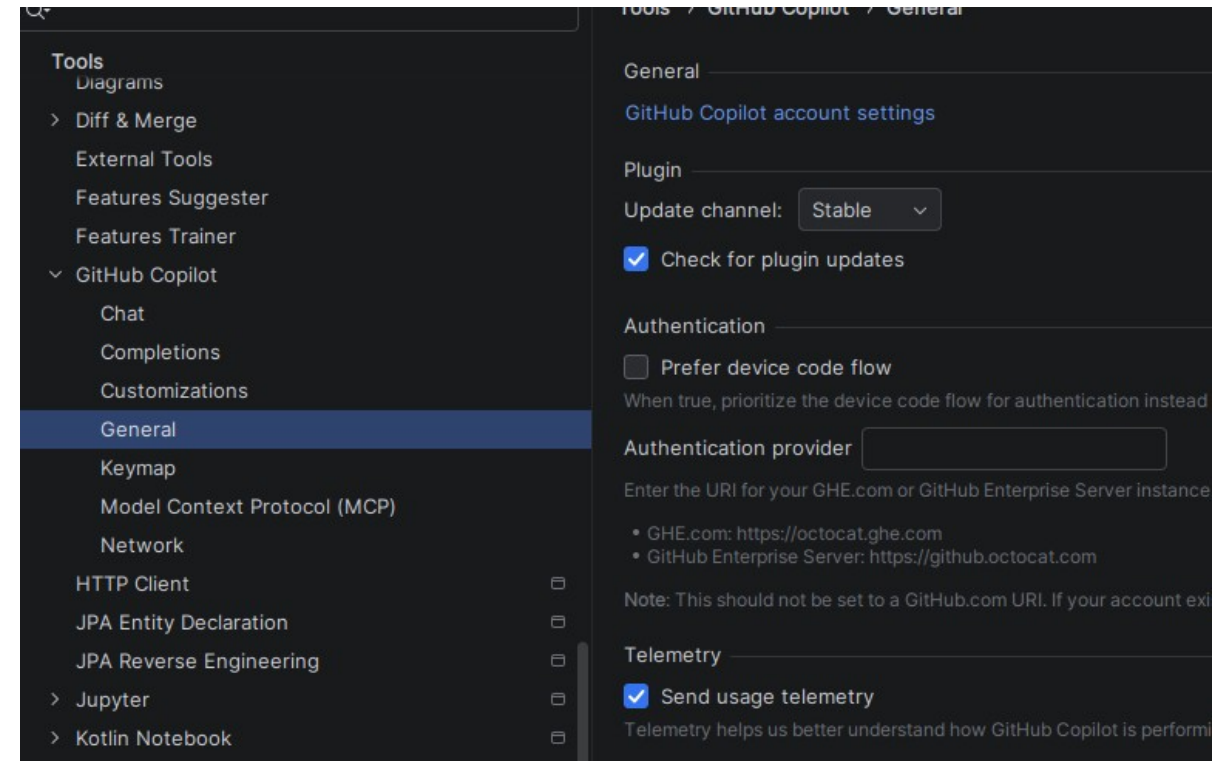
Copilot Integration

- Accessed via
 - [Settings](#) → [Tools](#) → [GitHub Copilot](#)
- Key settings
 - Enable/disable completions
 - Toggle inline suggestions globally or per language
 - Show completions automatically
 - Whether suggestions appear as you type or on demand
 - Duplication detection
 - Filter suggestions that closely match public code (if enabled by org)



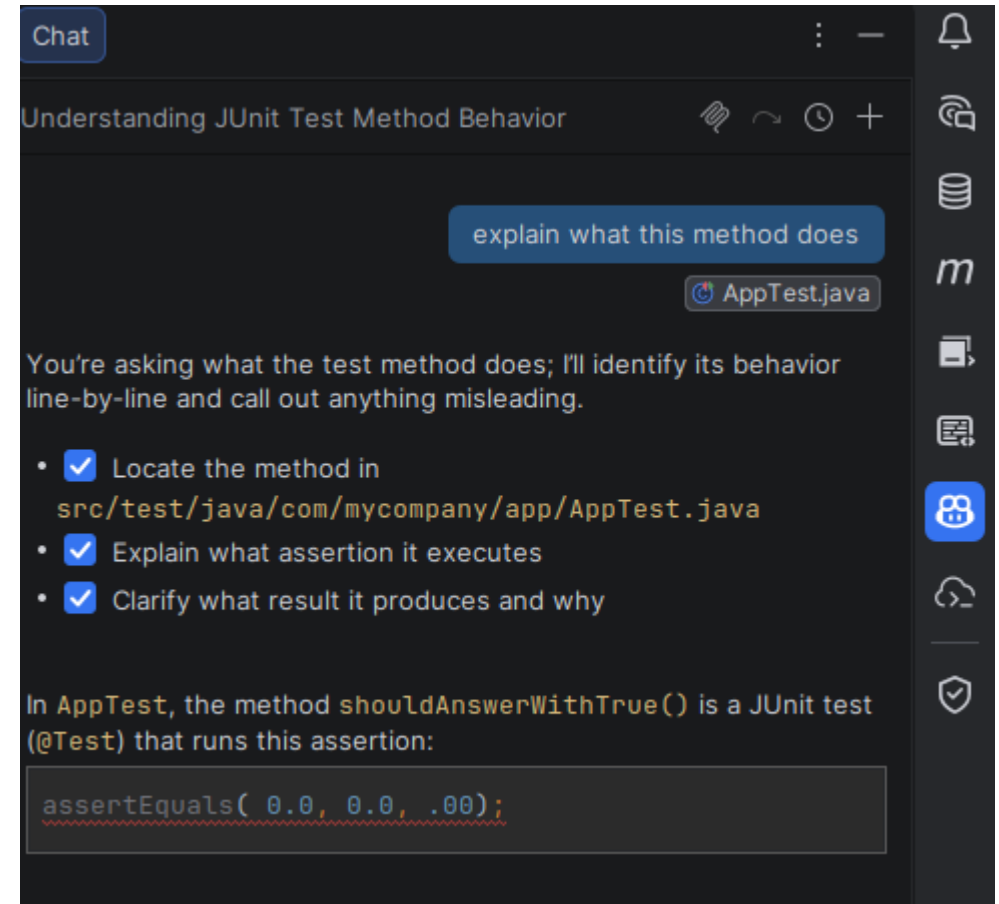
Copilot Integration

- Language-specific control
 - Completions can be enabled for Java but disabled for YAML or properties files
 - Useful if suggestions in configuration files create more noise than value
- Copilot Chat panel
 - Accessible via
 - [View](#) → [Tool Windows](#) → [GitHub Copilot Chat](#)
 - Can be docked to any side of the IDE



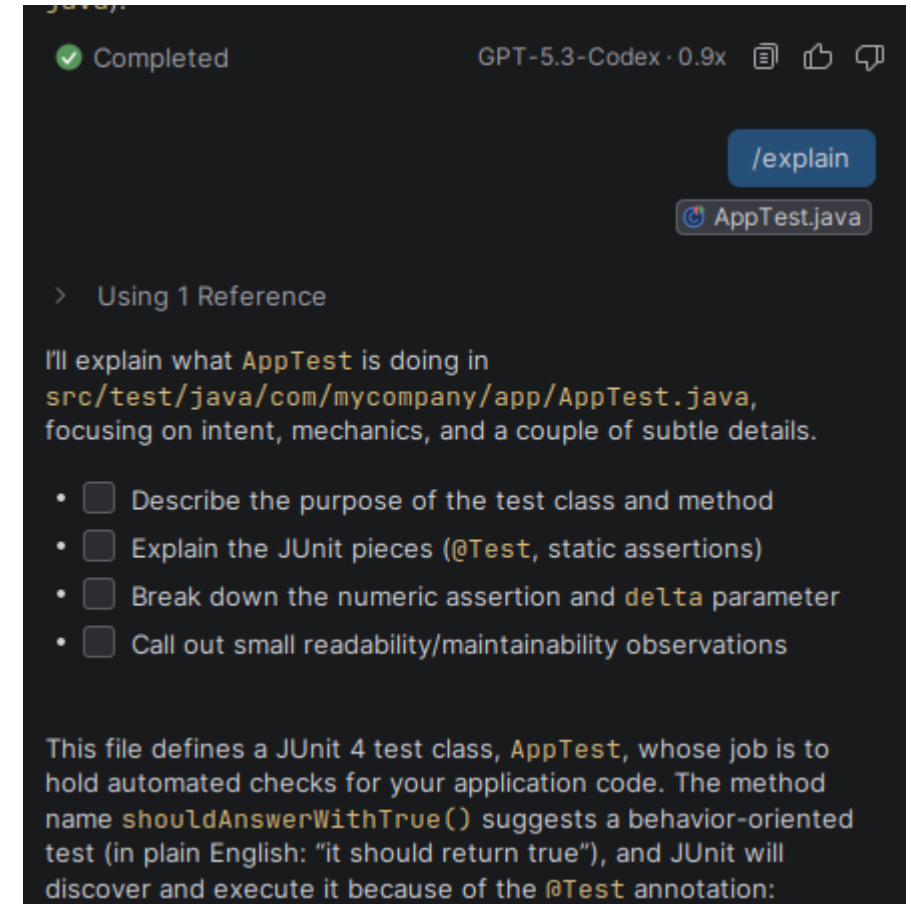
Copilot Chat Panel

- Conversational AI interface embedded in IntelliJ
 - Distinct from inline completions:
 - Completions suggest code as you type within the editor
 - Chat accepts free-form questions and returns detailed responses
- Chat is aware of
 - The file currently open in the editor
 - Selected code in the editor when you explicitly reference it
 - The project structure to a limited extent



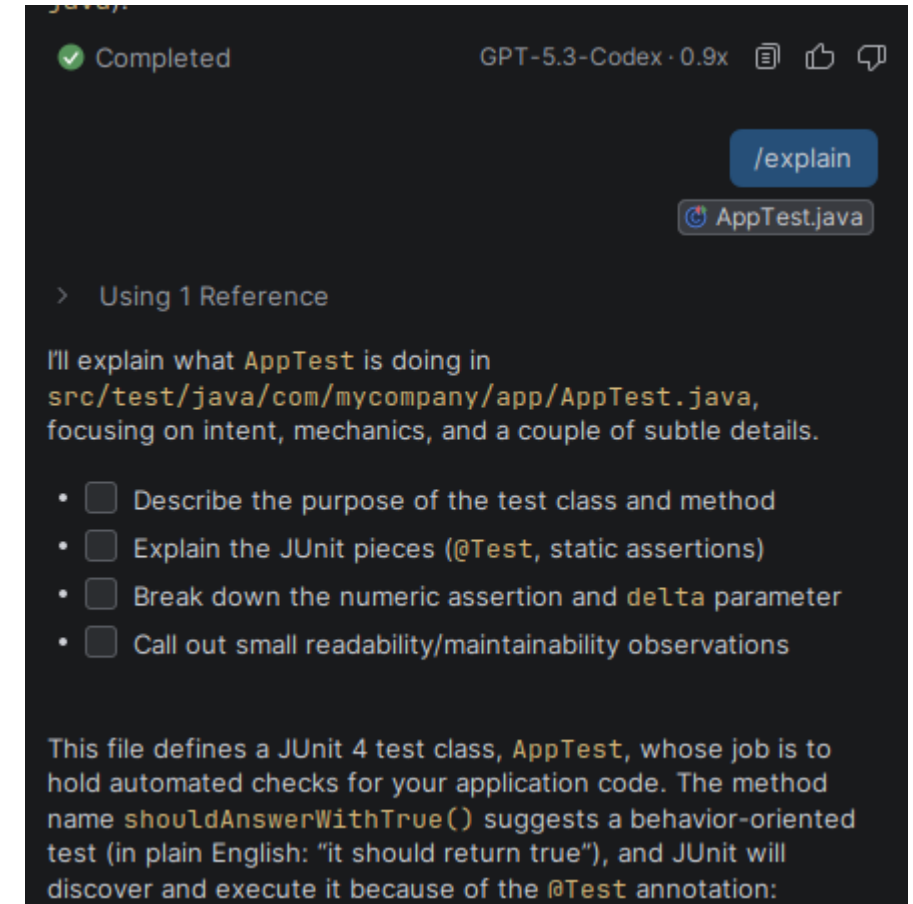
Copilot Chat Panel

- Slash commands in the Chat panel:
 - `/explain`
 - Explain selected code
 - `/fix`
 - Suggest a fix for a problem in selected code
 - `/tests`
 - Generate test cases for selected code
 - `/doc`
 - Generate Javadoc for selected code



Copilot Chat Panel

- Completions are passive
 - They appear as you write and are accepted or dismissed with a keystroke
- Chat is active
 - You initiate a conversation with a specific goal Developers transitioning from text-editor
 - In practice, Chat is where the most significant productivity gains come from for complex tasks
 - Explaining unfamiliar Spring Boot patterns
 - Generating a full test class for an existing service
 - Asking for a refactored version of a method that is too complex



Keyboard Shortcuts

- Common mistake
 - Using tab to accept a full suggestion
 - **Crtl-Right** for partial acceptance is often a more robust approach
 - Forces a more rigorous review of the suggestion
- Cycling through suggestions with **Alt+] and Alt+[**
 - Useful when the first suggestion is clearly wrong
 - Copilot typically has two to five alternatives for any given position

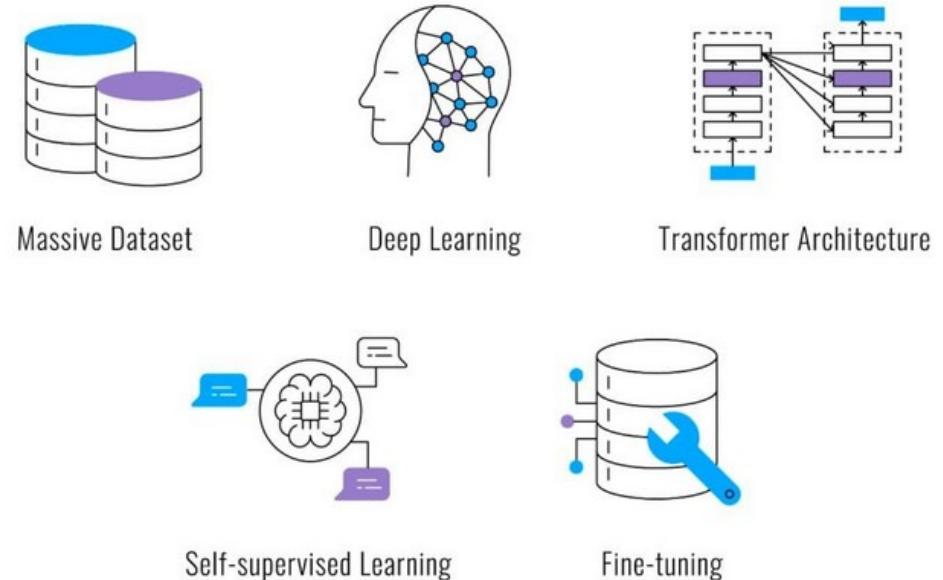
Action	Shortcut
Accept inline suggestion	Tab
Dismiss inline suggestion	Escape
Accept next word only	Ctrl+Right (Windows/Linux) / Cmd+Right (macOS)
Cycle to next suggestion	Alt+]
Cycle to previous suggestion	Alt+[
Trigger suggestion manually	Alt+\
Open Copilot Chat	Ctrl+Shift+C (configurable)
Explain selected code (Chat)	Select code → right-click → Copilot → Explain (Chat)

Understanding Copilot

GitHub Copilot

- AI-powered code completion tool
 - Built on a Large Language Model (LLM) trained primarily on source code
 - The underlying model is a descendant of the GPT model family
 - Trained on a large corpus of publicly available code from GitHub repositories, documentation, and technical text

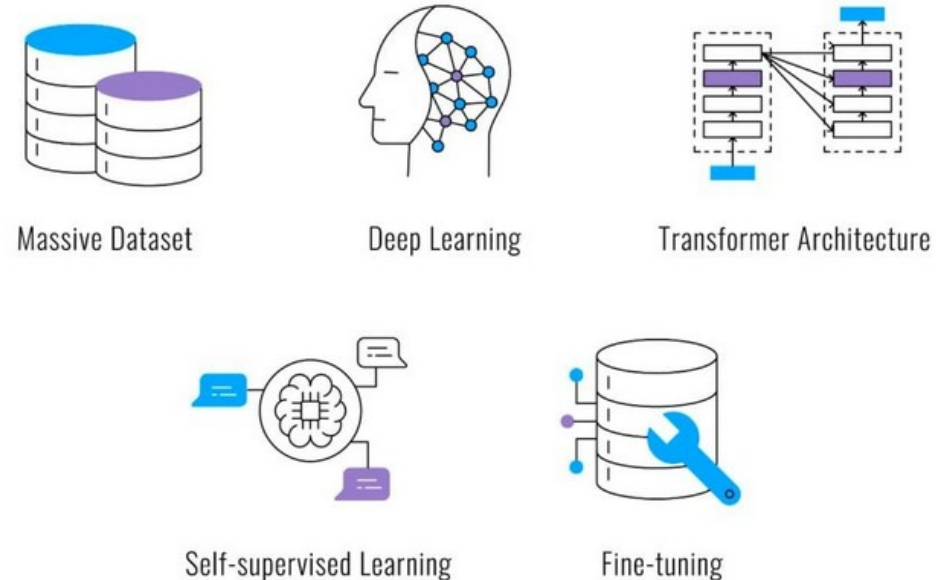
Large Language Models (LLM)



GitHub Copilot

- Copilot is a next-token prediction engine
 - Given a sequence of text consisting of your code, comments, and context
 - It predicts what text should come next
 - It does not understand your intent
 - It predicts what text is statistically likely to follow the text it has seen
 - It does not “understand” code
- It is not
 - A compiler or static analyzer
 - A search engine that retrieves existing code
 - A system that reasons about correctness

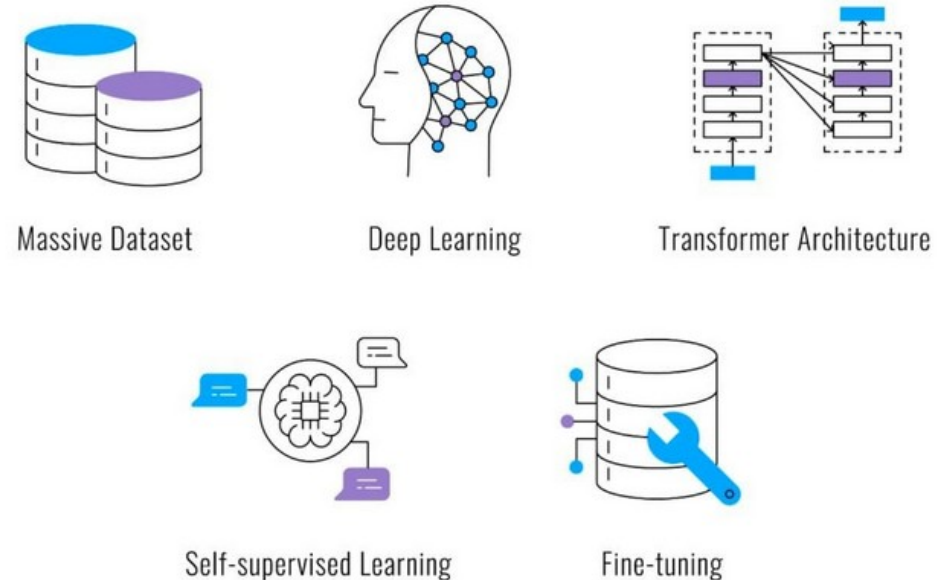
Large Language Models (LLM)



Training Data Foundation

- Copilot was trained on a large corpus of publicly available code
 - Billions of lines of code across many languages, with Java being heavily represented
 - Also includes Stack Overflow content, documentation, and technical articles
- Quality of suggestions correlates with
 - How common the pattern is in training data
 - How much relevant context exists in the current file and surrounding code

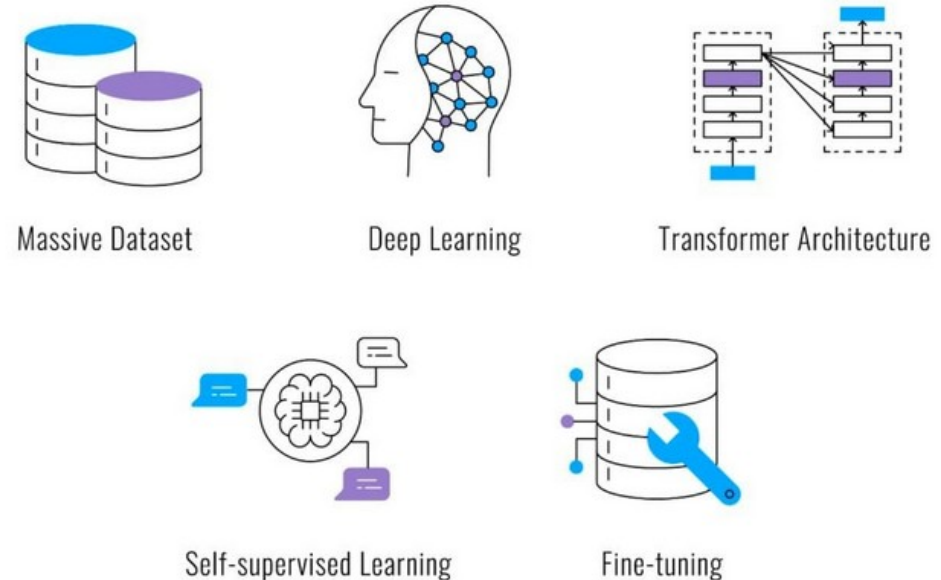
Large Language Models (LLM)



Training Data Foundation

- Implications of training data for Java and Spring Boot
 - Common patterns are well-represented
 - REST controllers, JPA repositories, service layers
 - Standard annotations are understood contextually
 - `@RestController`, `@Service`, `@Transactional`
 - Recent framework versions may be underrepresented
 - Organization-specific patterns and custom frameworks are not in the training data

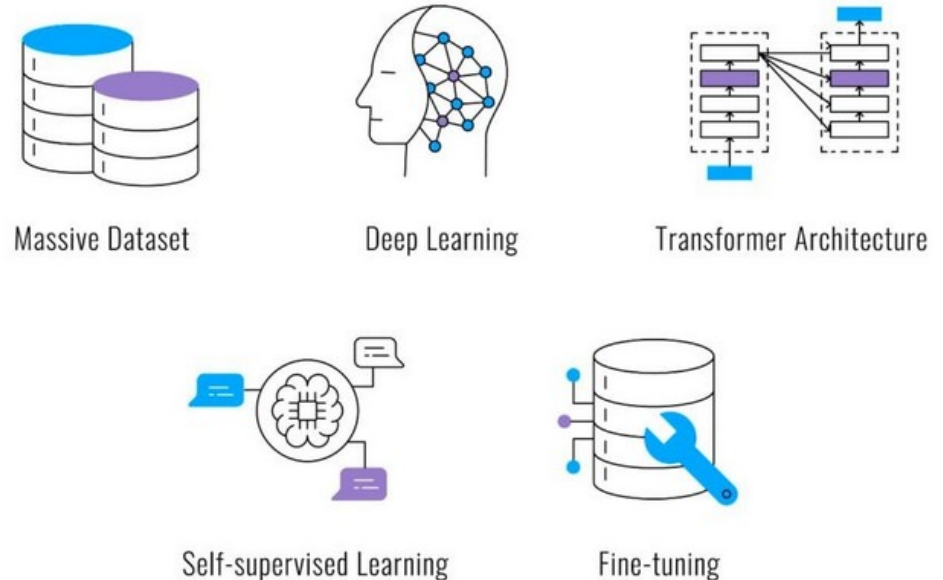
Large Language Models (LLM)



Training Data Foundation

- For Java and Spring Boot specifically
 - Copilot's training data coverage is strong for the standard patterns
 - CRUD operations
 - REST endpoint definitions
 - JPA query methods
 - Exception handling
 - Unit test structure
 - It performs less reliably on
 - Specialized domains
 - Custom security filter implementations
 - Complex Spring batch configurations
 - Oracle-specific SQL optimizations
 - Kafka consumer error handling with specific retry semantics

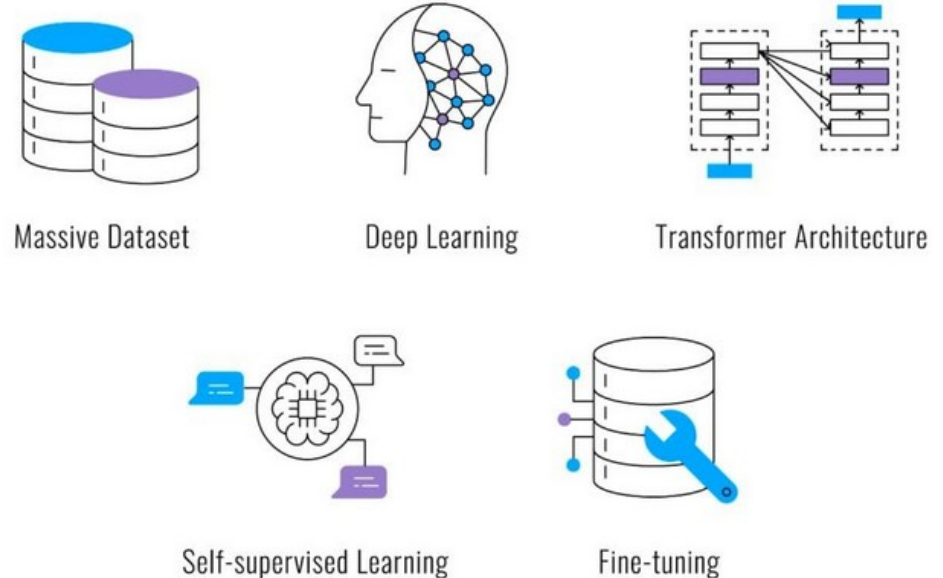
Large Language Models (LLM)



Training Data Foundation

- Where training coverage is weak
 - Copilot can still provide a useful starting point
 - But the developer's expert knowledge turn the generic suggestion into the correct implementation

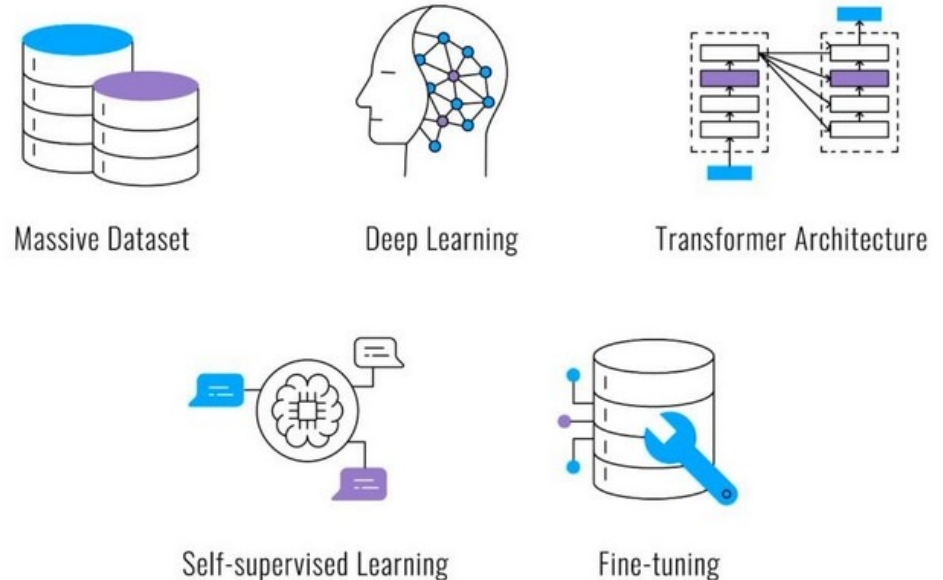
Large Language Models (LLM)



The Context Window

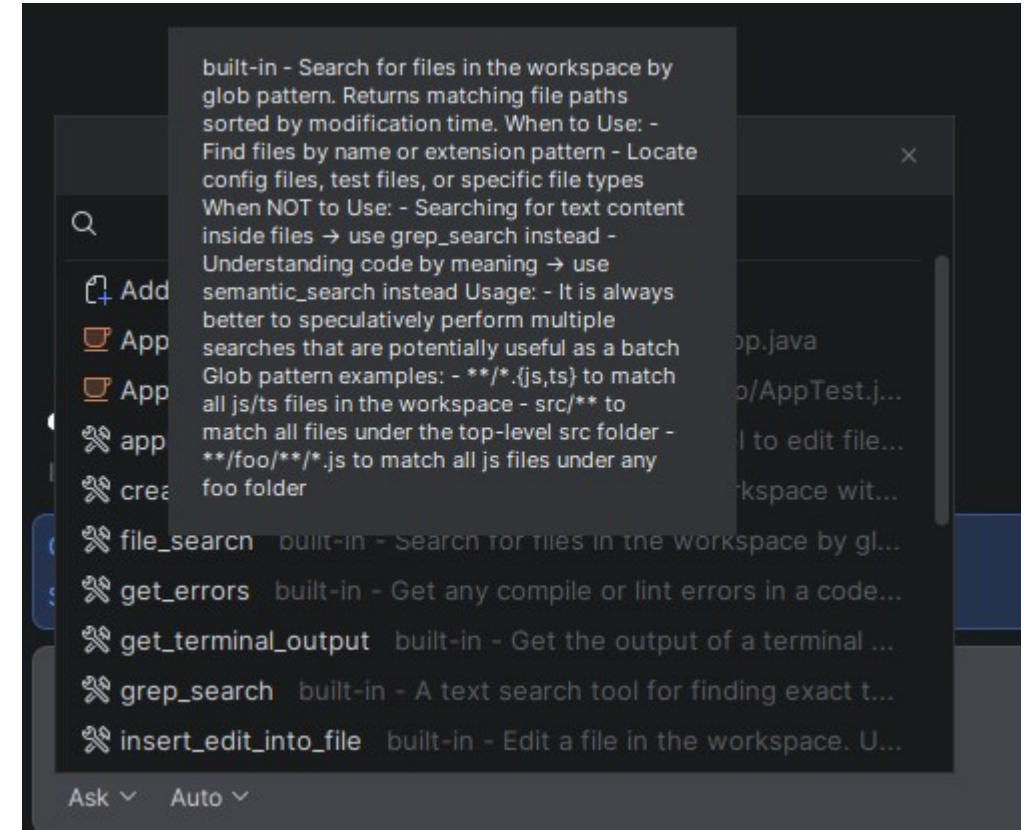
- Copilot's suggestions are determined by what it can see
 - Called the context window
 - Context includes
 - The current file
 - Everything above and below the cursor
 - The filename and path, used to infer intent
 - Recently opened files in the same IDE session
 - Selected code or text when using Chat

Large Language Models (LLM)



The Context Window

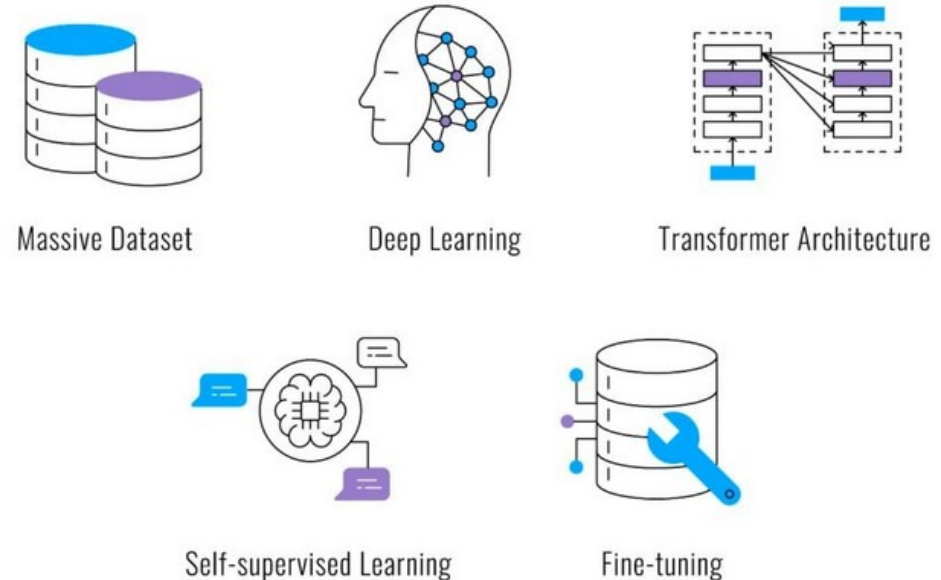
- Context
 - Does not automatically include
 - Other files in your project that you have not opened
 - Your application.yml configuration
 - Your database schema
 - Business rules that exist only in documentation or in someone's head
 - The quality of Copilot's output is directly constrained by the quality of its context
 - You can add artifacts to the context



Capabilities

- Well-suited tasks
 - Generating boilerplate
 - getters, constructors, equals/hashCode, toString
 - Scaffolding standard patterns
 - REST controllers, service classes, JPA repositories
 - Writing unit tests for methods with clear input/output contracts
 - Completing well-established algorithm patterns
 - Generating Javadoc from method signatures and parameter names
 - Translating a descriptive comment into a code implementation

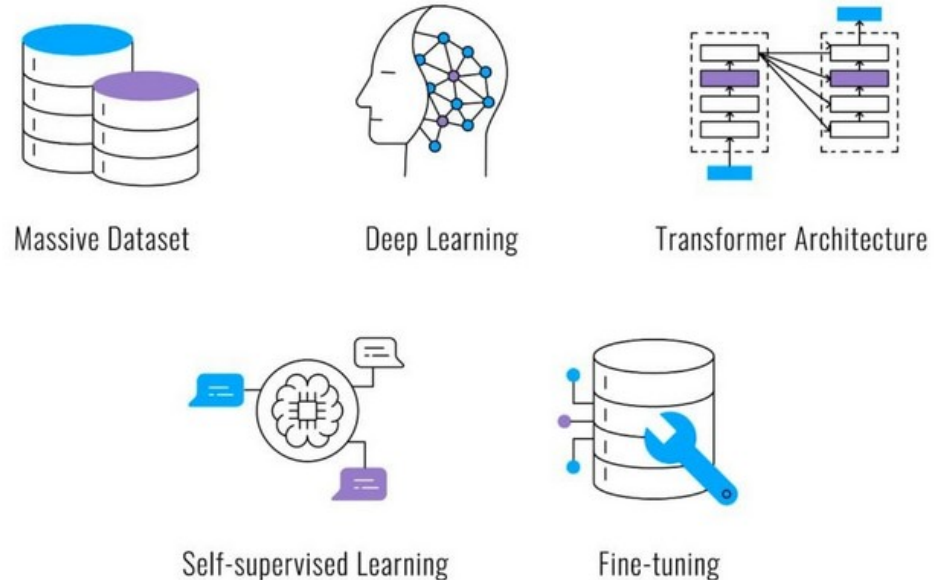
Large Language Models (LLM)



Capabilities

- Poorly-suited tasks
 - Reasoning about correctness in novel or domain-specific logic
 - Understanding cross-file dependencies it has not seen
 - Producing correct implementations of security-sensitive code without explicit guidance
 - Making architectural decisions
 - Replacing code review

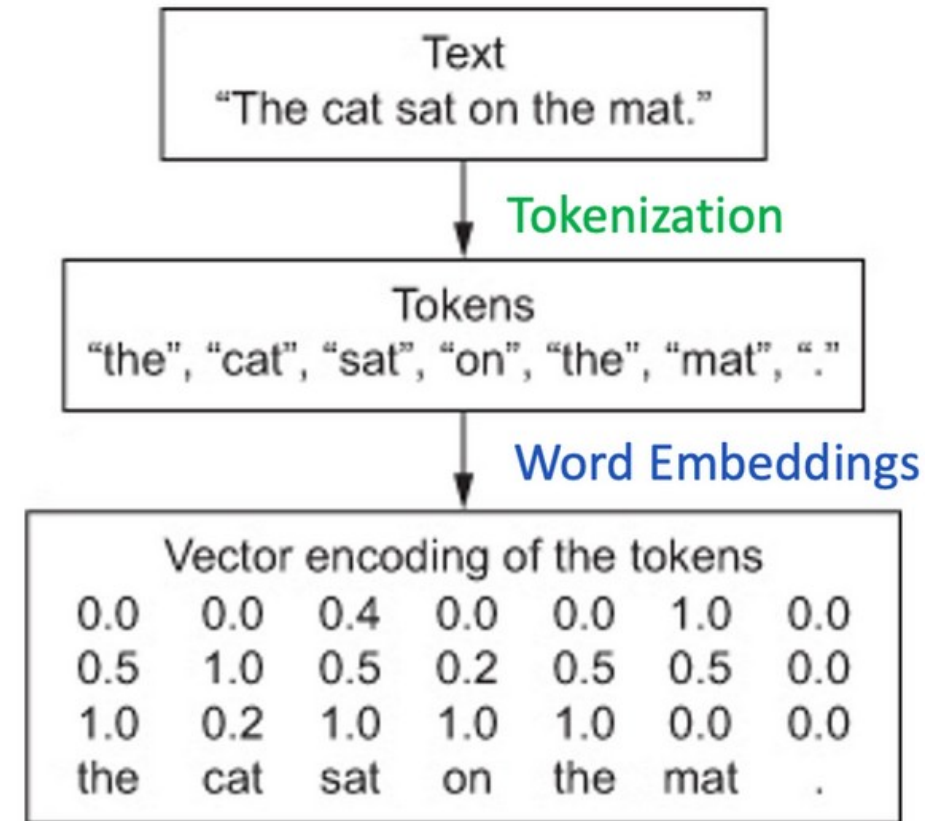
Large Language Models (LLM)



How Prompts Work

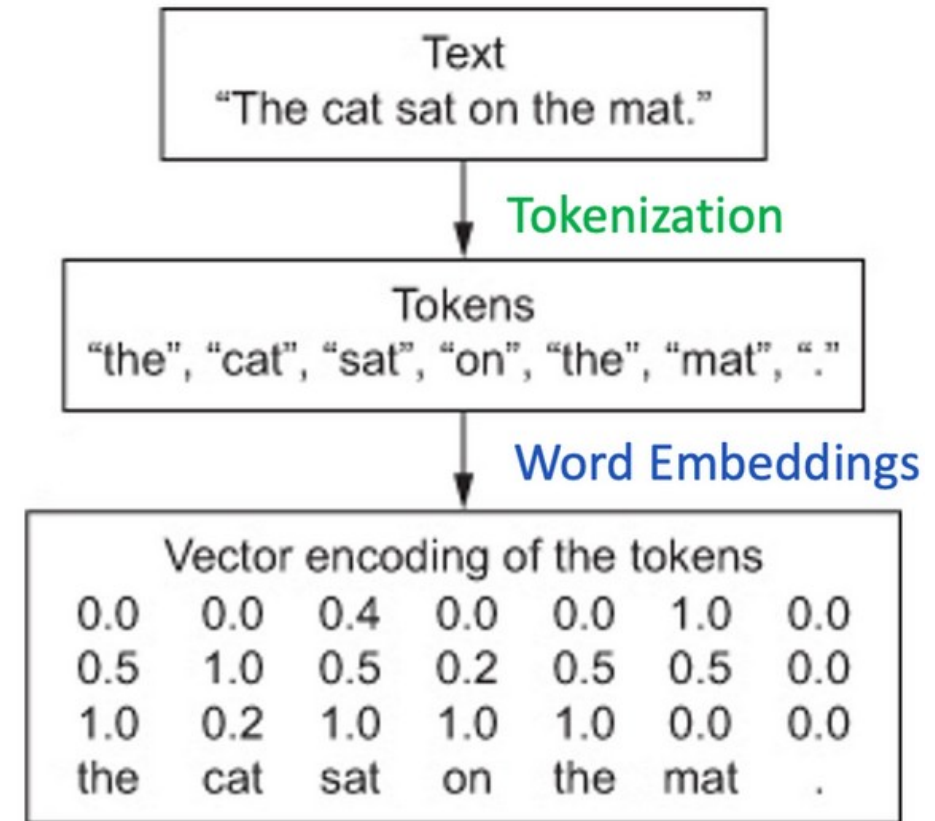
Tokens

- LLMs
 - Do not process characters, words, or lines
 - They process tokens
 - A token is roughly 3–4 characters of text on average
 - Common words are single tokens; rare words or identifiers may be multiple tokens
 - Code identifiers, keywords, punctuation, and whitespace are all tokenized



Tokens

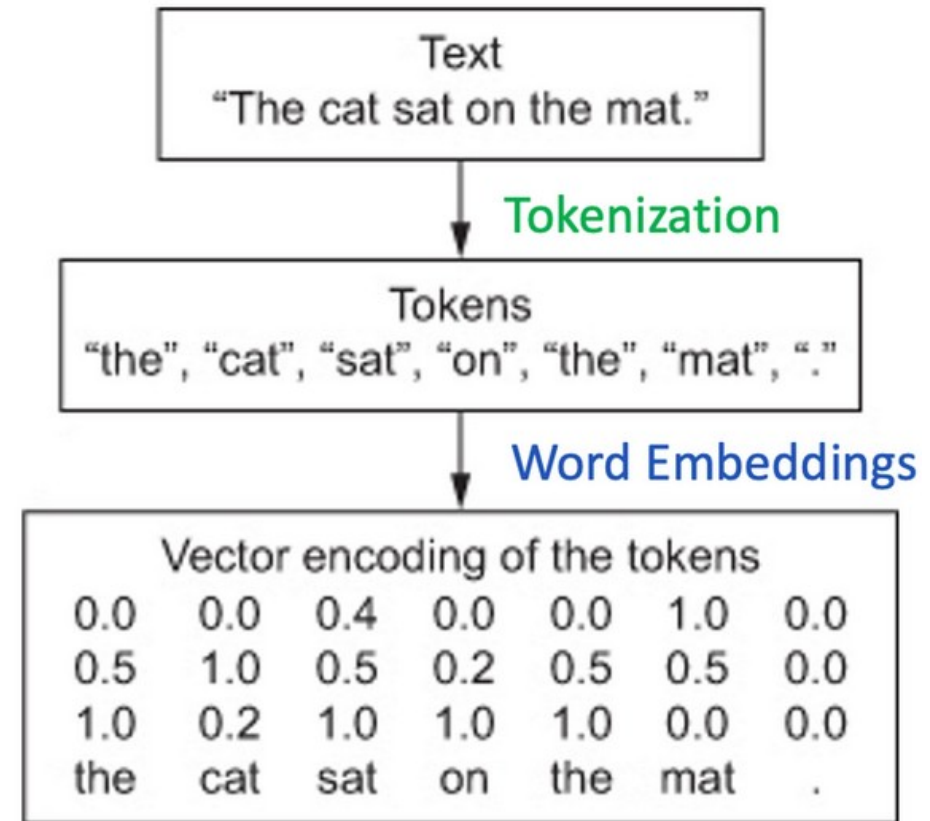
- LLMs
 - Copilot's context window is measured in tokens, not characters or lines
 - A file with 500 lines of Java code may consume 4,000–8,000 tokens
 - Copilot can only "see" the tokens that fit within its context window
 - Practical implication:
 - In a very large file, code far from the cursor may fall outside the context window
 - Copilot weighs tokens closer to the cursor more heavily than those further away



Tokens

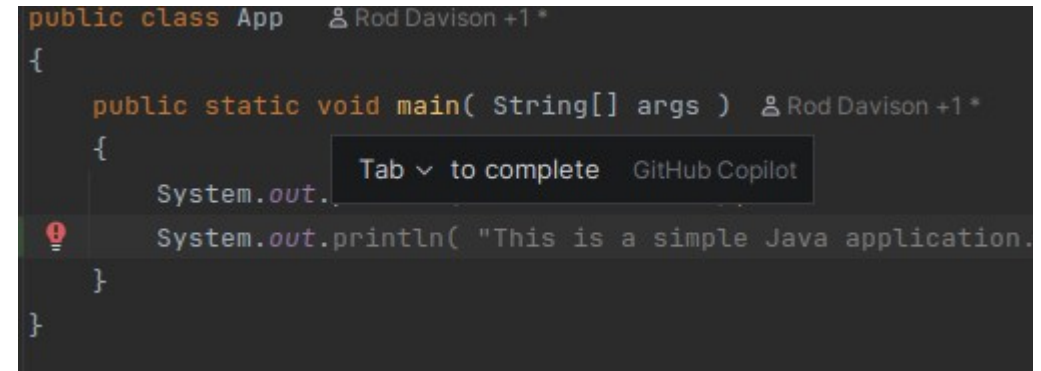
- Implications

- If a Java class is 800 lines long and the cursor is near the bottom
 - The class-level fields declared at the top may be partially or fully outside the context window by the time Copilot is generating a suggestion near the end of the file
- This is one reason to prefer smaller, well-focused classes
- A 400-line God class receives worse AI assistance than the same logic split into four 100-line focused classes
 - Each of which fits comfortably inside the context window



Completion

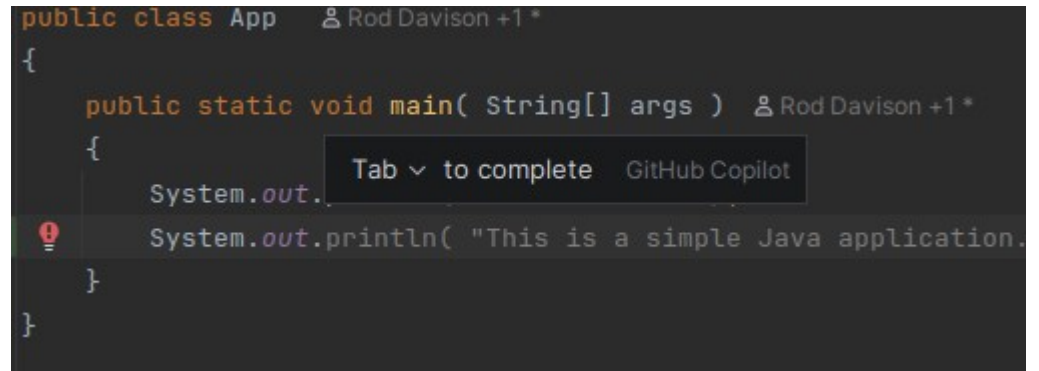
- When you pause typing
 - Copilot sends context to the model and receives a completion
- The process
 - IntelliJ plugin assembles the context
 - Current file, cursor position, recently opened files
 - Context is tokenized and sent to the Copilot model
 - The model is running on GitHub's servers
 - The model generates a probability distribution over possible next tokens
 - The most probable sequence of tokens is returned as the suggestion
 - The plugin renders this as greyed-out text in the editor



```
public class App  Rod Davison +1 *  
{  
    public static void main( String[] args )  Rod Davison +1 *  
    {  
        System.out.  
        System.out.println( "This is a simple Java application."  
    }  
}
```

Completion

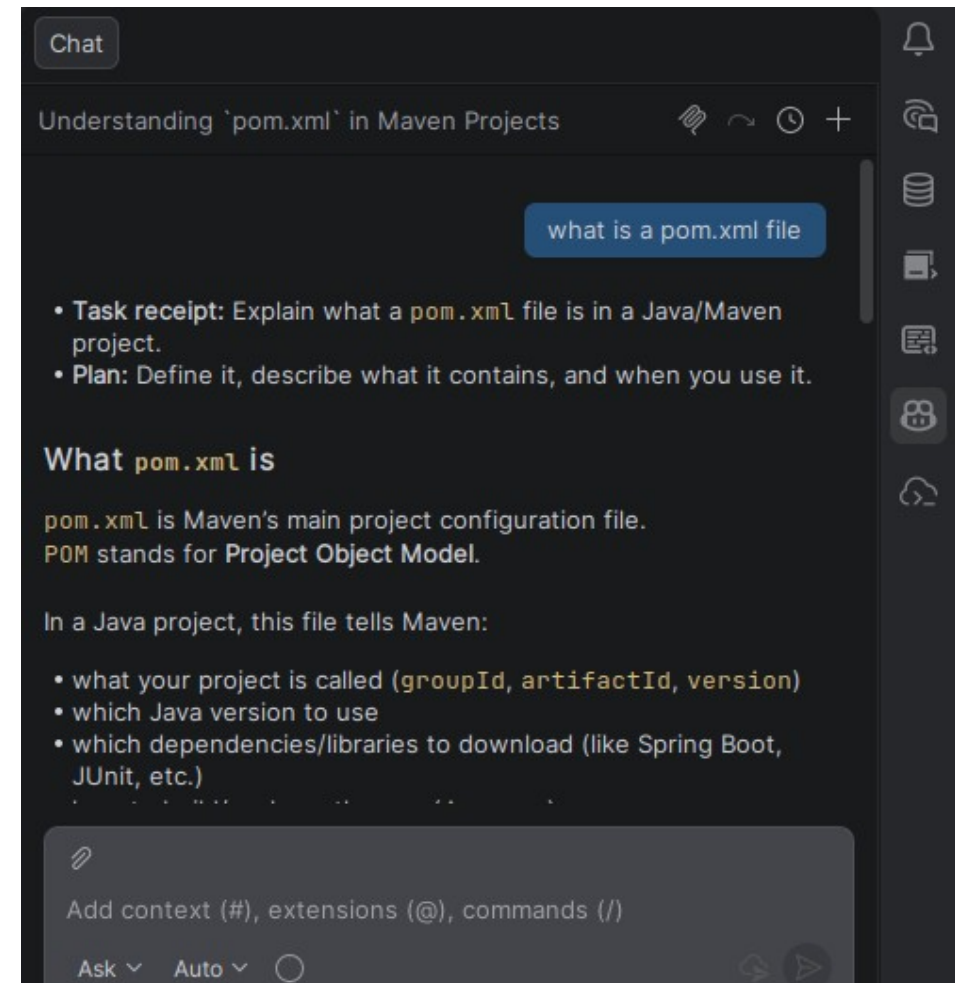
- The model does not run locally
 - Suggestions require an internet connection
 - Latency is typically under one second for inline completions



```
public class App  Ⓜ Rod Davison +1 *  
{  
    public static void main( String[] args )  Ⓜ Rod Davison +1 *  
    {  
        System.out.  
        System.out.println( "This is a simple Java application."  
    }  
}
```

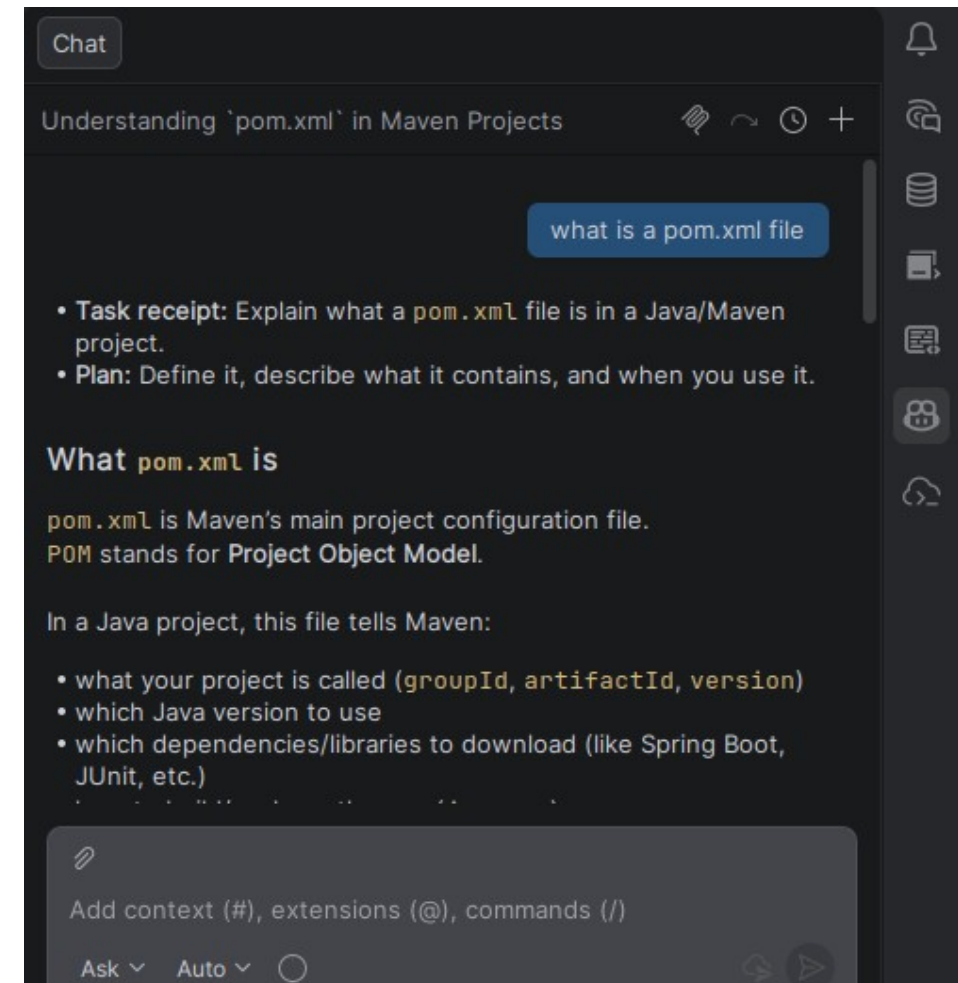
Chat Interactions

- Copilot Chat
 - Uses a conversational prompt-response model
 - You write a natural-language prompt
 - A question, instruction, or description
 - The model generates a response based on the prompt plus context
- The model sees
 - Your prompt text
 - The current file open in the editor
 - Automatically included
 - Any code you have explicitly selected or referenced with `#` syntax
 - The conversation history within the current Chat session



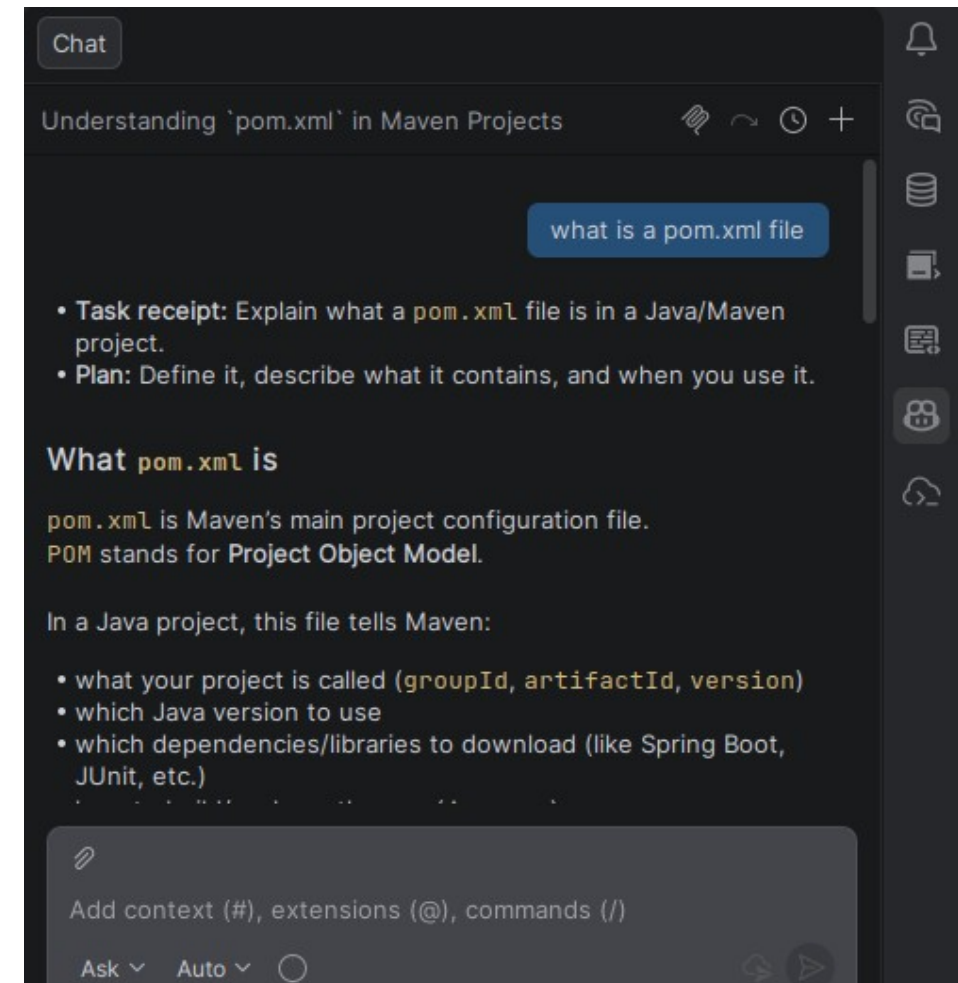
Chat Interactions

- Chat responses can include:
 - Natural-language explanations
 - Code blocks
 - Java, YAML, SQL, or any language
 - Mixed text and code



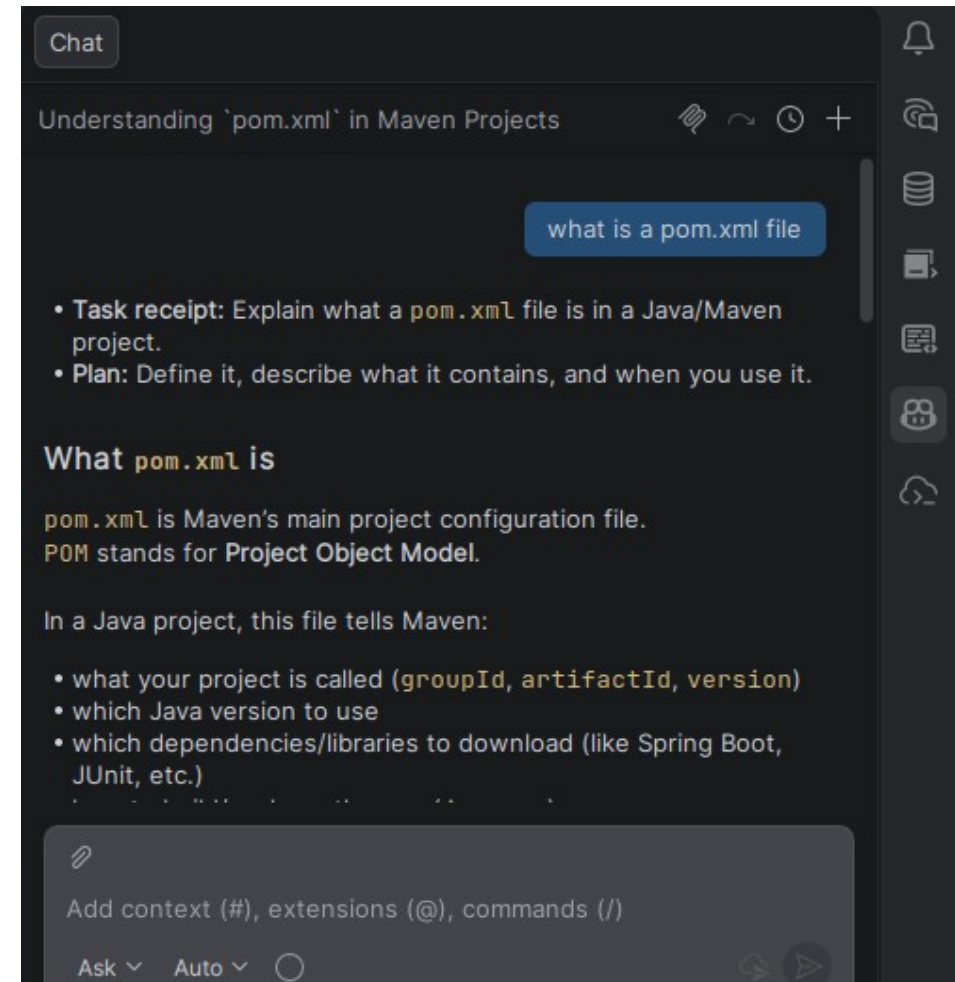
Chat Interactions

- Typing # in the Chat input
 - Opens a picker that lets you reference specific files, symbols, or even the entire project
 - For example
 - Typing "Refactor #OrderService to use the repository pattern"
 - Tells the model explicitly which file to focus on
 - Without the reference, the model uses whatever is in the active editor



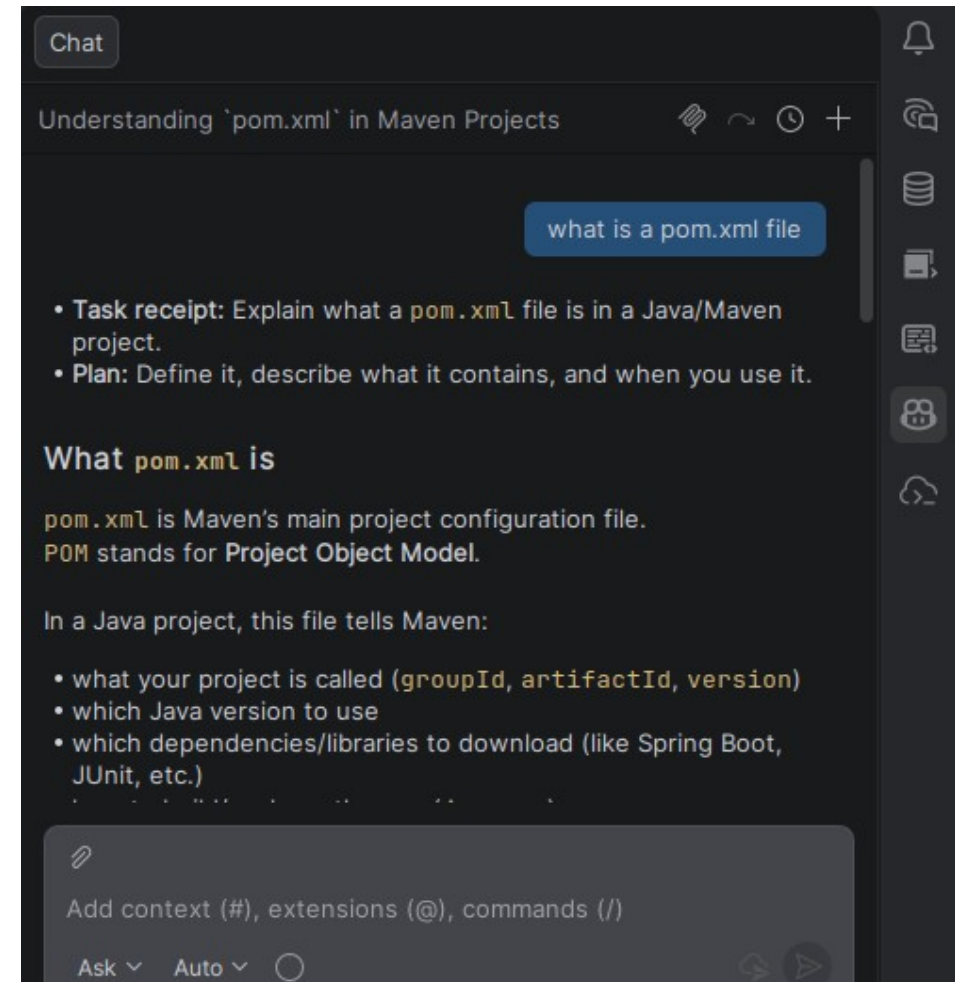
Same Prompt Different Outputs

- LLMs have a temperature parameter that controls output randomness
 - Higher temperature: more varied, creative outputs
 - Lower temperature: more deterministic, predictable outputs
 - Copilot is tuned for a moderate temperature appropriate for code generation



Same Prompt Different Outputs

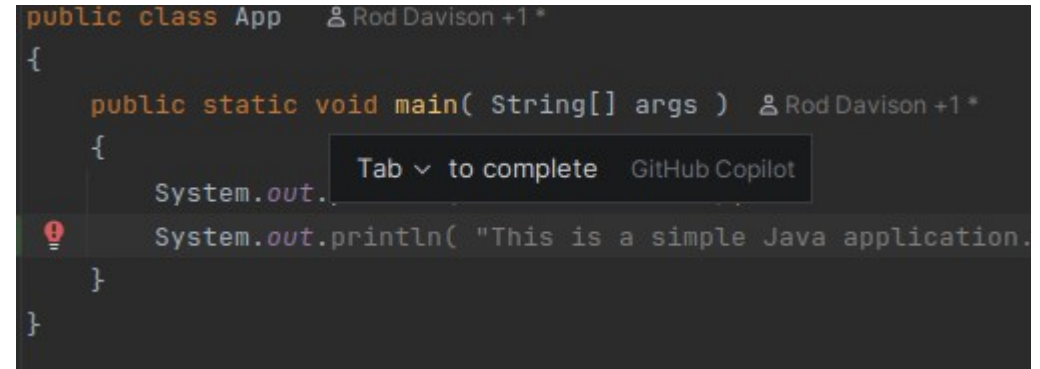
- Practical consequence
 - Running the same prompt multiple times may produce different results
 - Use `Alt+]` / `Alt+[` to cycle through alternative suggestions
 - In Chat, you can ask again or ask Copilot to try a different approach
 - This is not a bug
 - It reflects the probabilistic nature of the underlying model
 - It means never assume the first suggestion is the only possible suggestion



Using Copilot to Write and Validate Code

Inline Completions

- The primary Copilot interaction during active coding
- Workflow
 - Write a method signature or a descriptive comment
 - Pause while Copilot generates a suggestion as greyed-out text
 - Read the suggestion before accepting
 - Accept (**Tab**), accept word-by-word (**Ctrl+Right**), cycle alternatives (**Alt+]**), or dismiss (**Esc**)

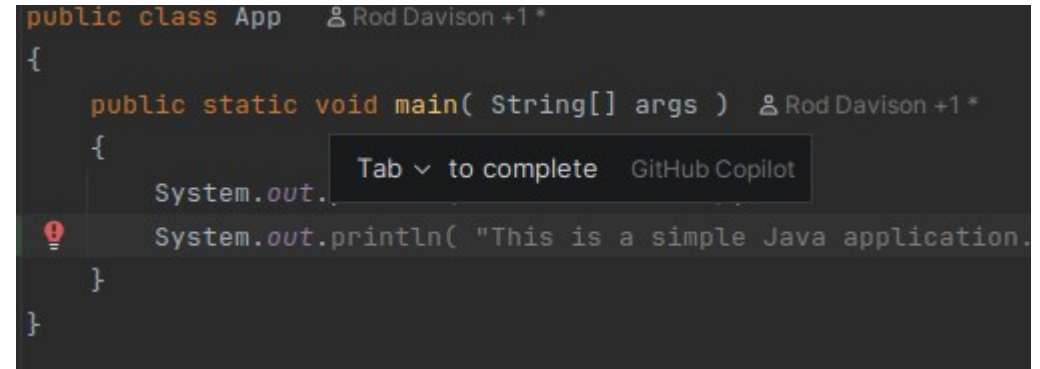


The screenshot shows a code editor with a Java class named 'App'. Inside the class, there is a 'main' method signature: 'public static void main(String[] args)'. A tooltip is visible over the 'main' method, displaying 'Tab v to complete' and 'GitHub Copilot'. Below the tooltip, a suggestion is shown in a greyed-out state: 'System.out.println("This is a simple Java application.")'. The suggestion is preceded by a red exclamation mark icon, indicating a warning or error. The code is written in a dark-themed editor.

```
public class App  Rod Davison +1 *  
{  
    public static void main( String[] args )  Rod Davison +1 *  
    {  
        System.out.  
        System.out.println( "This is a simple Java application."  
    }  
}
```

Inline Completions

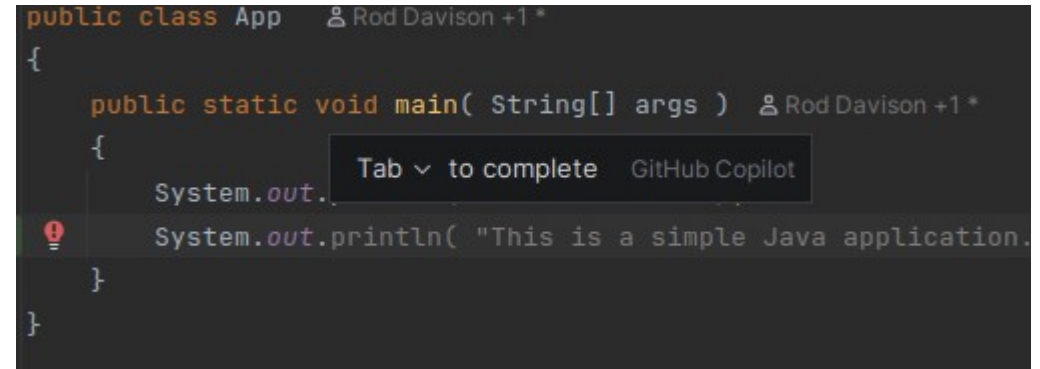
- Copilot is most effective when given clear signals
 - A descriptive method name that conveys intent
 - A typed return type that constrains the output
 - A Javadoc comment describing the expected behaviour
 - An existing similar method in the same class (pattern matching)



The screenshot shows a code editor with a Java class named `App`. Inside the class, there is a `main` method with the signature `public static void main( String[] args )`. The cursor is positioned at the end of the line `System.out.`. A tooltip is displayed above the cursor, showing the text `Tab to complete` and the GitHub Copilot logo. Below the tooltip, the text `System.out.println( "This is a simple Java application."` is visible, indicating the completion suggestion. The code is enclosed in curly braces, and the class name `App` is highlighted in orange.

Inline Completions

- Caveat
 - Tab completion in traditional IDEs is safe to accept without reading
 - Because the IDE is drawing from the resolved type graph
 - It can only offer valid completions
 - Copilot suggestions are probabilistic and can be wrong.
 - Treat every Copilot suggestion as a code review of code written by a fast but fallible junior developer
 - Accept or reject based on your understanding, not on its plausibility



```
public class App  Rod Davison +1 *  
{  
    public static void main( String[] args )  Rod Davison +1 *  
    {  
        System.out.  
        System.out.println( "This is a simple Java application."  
    }  
}
```

Comment-Driven Code

- Writing a descriptive comment
 - Describes what the code does
 - The comment acts as a specification
 - The more precise it is
 - The more accurate the suggestion
 - Useful for
 - Complex query methods that would require significant thought to write from scratch
 - Algorithm implementations where the logic can be stated clearly in plain language
 - Utility methods with well-defined input/output contracts

```
// Find all orders placed in the last 30 days with a total value
// greater than the specified threshold. Return sorted by date descending.
public List<Order> findRecentHighValueOrders(BigDecimal threshold) {
    // Copilot generates the method body here
}
```

Comment-Driven Code

- Inverts the traditional workflow
 - Instead of writing code and then documenting it
 - You write the documentation first and let Copilot write the code
- Secondary benefit beyond productivity
 - It forces you to articulate exactly what the method should do before writing it
 - Developers who struggle to write a clear comment describing a method's behaviour
 - Often discover that they have not yet fully thought through what the method should do

```
// Find all orders placed in the last 30 days with a total value
// greater than the specified threshold. Return sorted by date descending.
public List<Order> findRecentHighValueOrders(BigDecimal threshold) {
    // Copilot generates the method body here
}
```

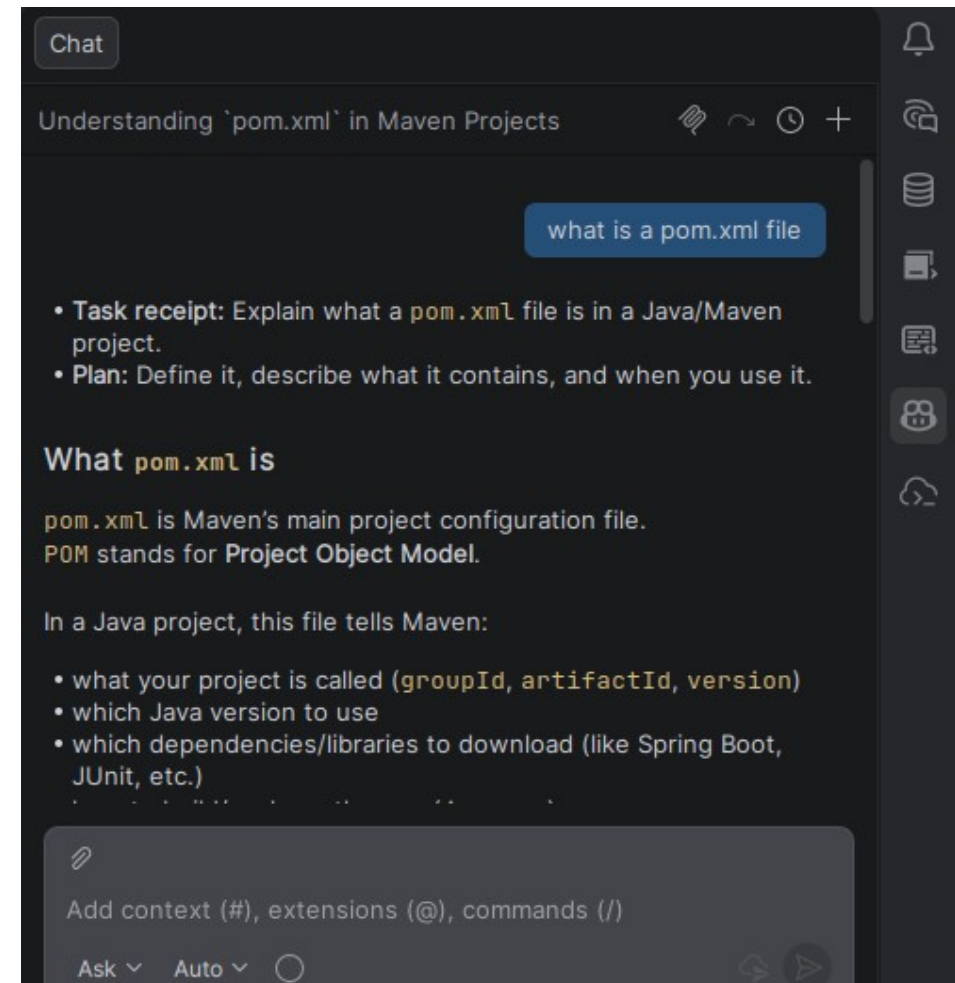
Comment-Driven Code

- The comment-first discipline
 - Lightweight version of test-driven development's *"specify before you implement"* principle
- In a Spring Data JPA context
 - A comment describing the query semantics often produces a correct `@Query` annotation or repository method name on the first suggestion

```
// Find all orders placed in the last 30 days with a total value
// greater than the specified threshold. Return sorted by date descending.
public List<Order> findRecentHighValueOrders(BigDecimal threshold) {
    // Copilot generates the method body here
}
```

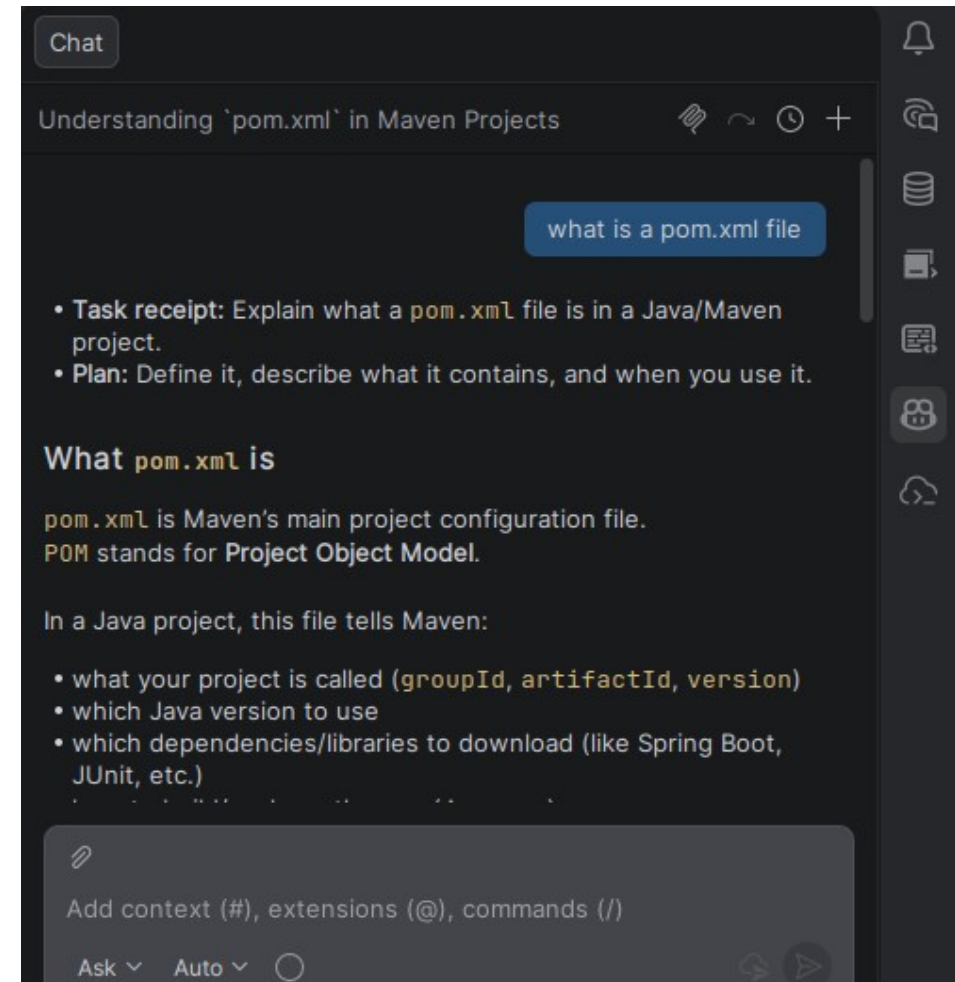
Writing Tests

- Approaches
 - Chat slash command
 - Select a method or class and write `/tests` in Chat
 - Inline
 - Open the test class, write the test method name, let Copilot complete the body
 - Comment-driven
 - Write
 - `// Test that findByEmail returns empty when user does not exist`
 - Generates tests reflect the method signature and naming, not the actual logic
 - The generated tests must be reviewed to verify they test the right behaviour
 - Tests that pass without catching the real behaviour are worse than no tests



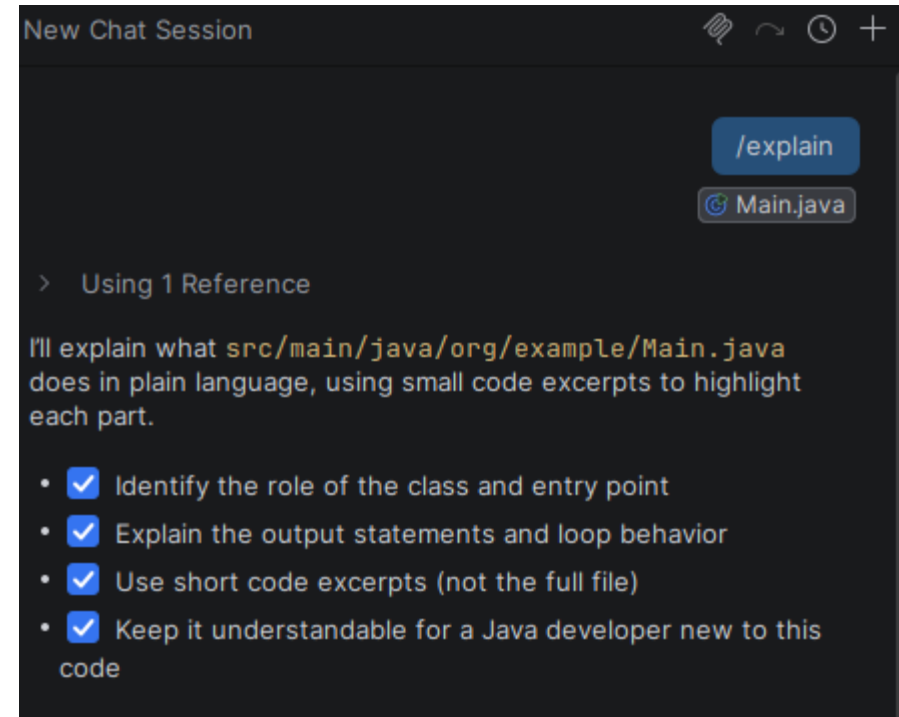
Writing Tests

- Copilot can generate a set of JUnit test methods
 - They compile, run, and pass
 - But that do not actually verify the contract of the method under test
 - Generates tests based on the method signature and name not on the implementation
 - A common pattern
 - Copilot generates an `assertEquals` that tests the exact value returned by the real implementation
 - Rather than testing what the value should be according to the specification
 - The result is a test that always passes



Explaining Unfamiliar Code

- Workflow
 - Select code you do not understand
 - [Right-click](#) → [GitHub Copilot](#) → [Explain This](#)
 - Or: type `/explain` in the Chat panel with code selected
- Useful for
 - Understanding a complex Spring Security filter chain
 - Decoding a multi-step Stream pipeline
 - Interpreting a JPA query with complex join conditions
 - Understanding the effect of an unfamiliar annotation



Fixing Errors and Refactoring

- Fix an error
 - Select
 - [error code](#) → [right-click](#) → [GitHub Copilot](#) → [Fix This](#)
 - [/fix](#) in Chat
 - Copilot proposes a correction and explains the change
 - Most useful for compilation errors and simple logic mistakes
- Refactoring via Chat
 - Select a method
 - Ask "Refactor this method to reduce its cyclomatic complexity"
 - Or: "Extract the validation logic into a separate method"
 - Or: "Rewrite this using Java Streams instead of a for loop"

Problem 1: Invalid `for` loop header syntax

All listed parser errors come from the same issue in the loop declaration: the condition section incorrectly redeclares `i` (`int i <= 5`) instead of just checking it (`i <= 5`). In Java, a `for` loop header has exactly three parts: initialization, condition, and update.

Before

```
for (int i = 1; int i <= 5; i++) {  
    System.out.println("i = " + i);  
}
```

After

```
for (int i = 1; i <= 5; i++) {  
    System.out.println("i = " + i);  
}
```

This single change resolves the grouped syntax/parsing errors and keeps the code simple and readable.

Since only one code change is required, a full rewritten file is not necessary.

Fixing Errors and Refactoring

- Important:
 - Refactoring via Copilot does not replace IntelliJ's semantic refactoring tools
 - IntelliJ's Rename, Extract Method, etc. are guaranteed to compile
 - Copilot's refactoring suggestions must be compiled and tested before they are trusted

Problem 1: Invalid `for` loop header syntax

All listed parser errors come from the same issue in the loop declaration: the condition section incorrectly redeclares `i` (`int i <= 5`) instead of just checking it (`i <= 5`). In Java, a `for` loop header has exactly three parts: initialization, condition, and update.

Before

```
for (int i = 1; int i <= 5; i++) {  
    System.out.println("i = " + i);  
}
```

After

```
for (int i = 1; i <= 5; i++) {  
    System.out.println("i = " + i);  
}
```

This single change resolves the grouped syntax/parsing errors and keeps the code simple and readable.

Since only one code change is required, a full rewritten file is not necessary.

Code Review and Validation

- Copilot Chat can be used as a first-pass review tool
 - Useful Chat prompts for validation:
 - "Are there any edge cases this method does not handle?"
 - "What happens if the input list is null or empty?"
 - "Is this implementation thread-safe?"
 - "Does this code have any potential resource leaks?"
 - "What are the security implications of this approach?"
 - Copilot's analysis is pattern-based and may miss issues that require runtime reasoning
- This is a supplement to, not a replacement for
 - IntelliJ's static inspections
 - Checkstyle rules
 - SAST tools (Checkmarx)
 - Human code review

Code Review and Validation

- Valuable use of Copilot for validation
 - Asking targeted questions about specific concerns
 - "Are there edge cases this method doesn't handle?"
 - Is a better prompt than
 - "Review this code"
 - Because it directs attention to a specific class of issue
 - Copilot answers to these questions are starting points for human analysis
 - An answer of "this appears thread-safe" from Copilot is an invitation to verify that claim, not a confirmation

Code Review and Validation

- What Copilot cannot validate
 - Copilot cannot verify correctness against requirements it has never seen
 - Business rules stored in external documentation
 - Non-functional requirements (performance thresholds, SLA constraints)
 - Compliance requirements not visible in the code
 - Copilot cannot detect all security vulnerabilities
 - It may generate vulnerable code while reviewing a prompt about security
 - It is not a substitute for dedicated SAST/DAST tooling (Checkmarx)
 - Copilot output is not subject to organizational policy or standards it has not been given
 - It does not know your team's Checkstyle rules unless you provide them
 - It does not know your organization's approved dependency list

Code Review and Validation

- Quality gates
 - IntelliJ inspections, Checkstyle, JUnit test coverage, SAST scanning with Checkmarx
 - Are not replaceable by Copilot
 - They are deterministic, enforceable, and auditable
- Copilot
 - Is probabilistic, advisory, and not auditable in the same way
- Replacing formal quality gates with Copilot is any error
 - The correct mental model
 - Copilot accelerates the developer
 - The quality gate system ensures the output meets organizational standards regardless of how it was produced

Code Review and Validation

- Quality gates
 - IntelliJ inspections, Checkstyle, JUnit test coverage, SAST scanning with Checkmarx
 - Are not replaceable by Copilot
 - They are deterministic, enforceable, and auditable
- Copilot
 - Is probabilistic, advisory, and not auditable in the same way
- Replacing formal quality gates with Copilot is any error
 - The correct mental model
 - Copilot accelerates the developer
 - The quality gate system ensures the output meets organizational standards regardless of how it was produced

Prompt Engineering

Prompt Engineering

- Prompt engineering is the practice of designing inputs to an LLM to maximize output quality
 - it is closer to writing a precise specification or a clear brief by considering the prompt as a specification
 - Key insight: the model can only work with what it is given
 - Vague input → generic, often wrong output
 - Precise, contextual input → specific, often useful output
- Prompt engineering for code has two forms:
 - Inline context engineering
 - Shaping the code and comments the model can see
 - Chat prompt construction
 - Writing explicit, detailed prompts in the Chat panel

Prompt Engineering

- Principle 1: Be specific about intent
 - State what you want, why you want it, and any relevant constraints
 - Vague prompt:
 - "Write a method to get users"
 - Specific prompt:
 - "Write a Spring Data JPA repository method that retrieves all active users whose email domain matches a given string, returns a paginated result, and sorts by last name ascending"
- Specificity applies to
 - The input type and constraints
 - The expected return type
 - Edge case handling (null input, empty result, invalid argument)
 - The framework or library to use

Prompt Engineering

- Principle 2: Provide context explicitly
 - Do not assume Copilot can infer context it has not seen
 - Techniques for providing context
 - Open the relevant files before asking a Chat question (they enter the context window)
 - Reference specific files or symbols using # in Chat
 - Include relevant constraints in the prompt: "This service is called from multiple threads concurrently"
 - State the framework version: "Using Spring Boot 3.2 and Spring Security 6"
 - Paste the relevant interface or base class into Chat when asking about an implementation
 - The more relevant context the model has, the less it relies on generic patterns

Prompt Engineering

- Principle 3: Use the code as context
 - The code in the current file is the most powerful context available
 - Name things deliberately
 - `findActiveOrdersByCustomerId` is better context than `getOrders`
 - `CustomerValidationException` is better context than `AppException`
 - Write the method signature before asking Copilot to complete the body
 - The return type, parameter types, and parameter names all signal intent
 - Write a Javadoc comment before the method
 - Copilot treats Javadoc as a specification to implement
 - Use `@param` and `@return` tags: they constrain output effectively

Prompt Engineering

- Principle 4: Iterate, do not accept and move on
 - Treating the first suggestion as the final answer is the most common Copilot misuse pattern
 - Iteration strategies
 - Cycle through alternatives: Alt+] / Alt+[
 - Dismiss and retype with a more specific comment
 - Accept a partial suggestion, then ask Copilot to continue from the new position
 - In Chat: ask "Can you improve this to also handle the case where X?"
 - Iteration produces better results than re-prompting from scratch
 - The conversation history is part of the context
 - Follow-up prompts can refine rather than replace

Prompt Engineering

- Principle 5: Validate all output
 - Every piece of Copilot output must be validated before it is committed to the codebase
- Validation checklist for generated code
 - Does it compile?
 - Copilot can generate code that references non-existent methods
 - Does it do what I asked?
 - Read the code, not just the structure
 - Are edge cases handled?
 - Null inputs, empty collections, boundary values
 - Does it follow project conventions?
 - Naming, error handling, logging patterns
 - Does it pass the tests?
 - Run the test suite; write tests if none exist
 - Would this survive a code review?
 - Apply the same standard you would to any committed code

Prompt Engineering

- Principle 6: Avoid sensitive data in prompts
 - Copilot Chat prompts are sent to GitHub's servers
 - Do not include real credentials, connection strings, or secrets in prompts
 - Do not include personally identifiable information or sensitive business data
 - Practical rules
 - Use placeholder values in examples: "db_password_here", "customer-id-placeholder"
 - Ask conceptual questions rather than pasting data: "How do I handle a JWT with these claims?" not "Here is a real JWT token..."
 - Review your organization's Copilot usage policy for specific guidance on what data classifications are permitted
 - Code containing real infrastructure details (IP addresses, internal hostnames) should also be treated with caution

Prompt Patterns

- Pattern 1: Specify the layer
 - "Write a Spring @Service method that..." vs. "Write a method that..."
 - The Spring annotation in the prompt activates Spring-specific patterns
- Pattern 2: State exception handling intent
 - "...and throw a `ResourceNotFoundException` if not found" produces correct exception handling
 - Without this, Copilot may return `null` or an empty `Optional` silently

Prompt Patterns

- Pattern 3: Request tests alongside implementation
 - "Write the implementation and a corresponding JUnit 5 test class with Mockito"
 - Produces implementation and test in one step
- Pattern 4: Ask for explanations with code
 - "Write the implementation and add inline comments explaining each step"
 - Inline comments allow faster review and learning simultaneously

Hallucination

- Hallucination in LLMs
 - Generating confident, plausible-sounding output that is factually incorrect
 - In code generation, hallucination appears as
 - Calling methods that do not exist on a class or library
 - Referencing API signatures that were changed in a different version
 - Generating import statements for packages that do not exist
 - Asserting incorrect behaviour for a framework or library
 - Detection
 - The compiler is the first line of defence, non-existent methods cause compilation errors
 - IntelliJ's inspections highlight unresolved references immediately
 - Code review against documentation catches API misuse
 - Response: reject the suggestion and re-prompt with the more specific API information

Takeaways

- Copilot is a prediction engine, not a knowledge base or reasoning system
 - Quality of output is determined by quality of context
- Integration in IntelliJ provides two modes: inline completions and Chat
 - Both have distinct use cases and complement each other
- Prompts are specifications: precision and context determine output quality
 - Six prompt engineering principles
 - Be specific about intent
 - Provide context explicitly
 - Use the code itself as context
 - Iterate rather than accept and move on
 - Validate all output
 - Avoid sensitive data in prompts
 - Validation is non-negotiable
 - Copilot output must meet the same standard as human-written code

Questions?

